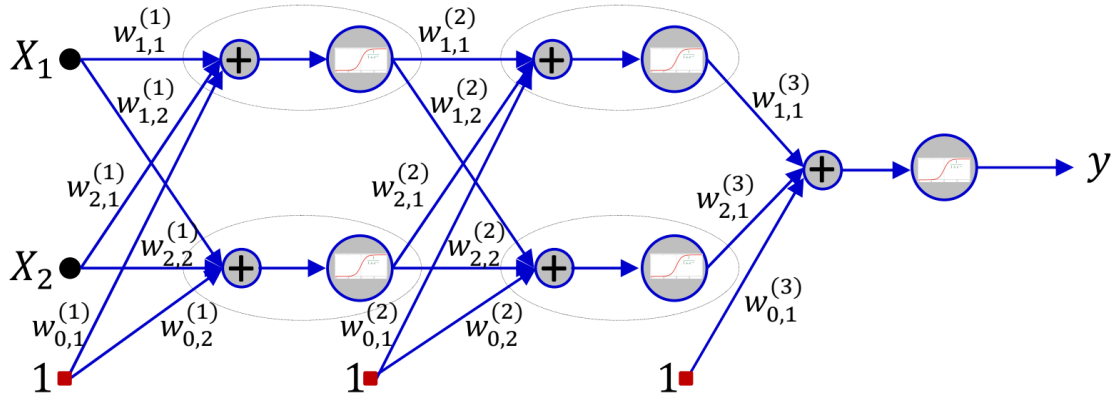


Backpropagation Derivation

3-Layer Network — Sigmoid Activations, Binary Cross-Entropy Loss



1. Network and Notation

The network has two inputs, two neurons in hidden layer 1, two neurons in hidden layer 2, and one output neuron in layer 3 (matching the diagram). Every neuron applies the sigmoid activation function σ to an affine transform z .

1.1 Pre-activation and activation

- For layer l , neuron j :

$z_j^{(l)} =$ affine transform (weighted sum + bias) into neuron j

$y_j^{(l)} = \sigma(z_j^{(l)}) \leftarrow$ activation / output of that neuron

So every layer computes z first, then applies the sigmoid to get y — exactly as in the diagram, and y is used (not “ a ”) for every neuron’s output at every layer, including the network’s final output.

1.2 Inputs as “layer 0”

$$y_1^{(0)} = x_1, \quad y_2^{(0)} = x_2$$

1.3 Weights

- $w_{i,j}^{(l)} =$ weight from neuron i in layer $l-1$ to neuron j in layer l
- $w_{0,j}^{(l)} =$ bias feeding into neuron j in layer l (from the constant input “1”)

1.4 Network output

$$y = y_1^{(3)} \text{ (the single output neuron of layer 3)}$$

2. Forward Pass

2.1 Layer 1 (hidden)

$$\begin{aligned}
 z_1^{(1)} &= w_{1,1}^{(1)}y_1^{(0)} + w_{2,1}^{(1)}y_2^{(0)} + w_{0,1}^{(1)} \\
 y_1^{(1)} &= \sigma(z_1^{(1)}) \\
 z_2^{(1)} &= w_{1,2}^{(1)}y_1^{(0)} + w_{2,2}^{(1)}y_2^{(0)} + w_{0,2}^{(1)} \\
 y_2^{(1)} &= \sigma(z_2^{(1)})
 \end{aligned}$$

2.2 Layer 2 (hidden)

$$\begin{aligned}
 z_1^{(2)} &= w_{1,1}^{(2)}y_1^{(1)} + w_{2,1}^{(2)}y_2^{(1)} + w_{0,1}^{(2)} \\
 y_1^{(2)} &= \sigma(z_1^{(2)}) \\
 z_2^{(2)} &= w_{1,2}^{(2)}y_1^{(1)} + w_{2,2}^{(2)}y_2^{(1)} + w_{0,2}^{(2)} \\
 y_2^{(2)} &= \sigma(z_2^{(2)})
 \end{aligned}$$

2.3 Layer 3 (output)

$$\begin{aligned}
 z_1^{(3)} &= w_{1,1}^{(3)}y_1^{(2)} + w_{2,1}^{(3)}y_2^{(2)} + w_{0,1}^{(3)} \\
 y &= y_1^{(3)} = \sigma(z_1^{(3)})
 \end{aligned}$$

3. Loss Function

Binary cross-entropy, with true label $t \in \{0, 1\}$ and network prediction y :

$$L = -[t \log y + (1 - t) \log(1 - y)]$$

4. Two Facts Used Throughout

4.1 Sigmoid derivative

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)) = y \cdot (1 - y)$$

(where y is that same neuron's own output — the derivative of a sigmoid can always be written in terms of its own output).

4.2 BCE derivative with respect to its input y

$$\partial L / \partial y = -t/y + (1-t)/(1-y) = (y - t) / [y(1-y)]$$

5. Backward Pass

Define the error signal at each neuron's pre-activation:

$$\delta_f^{(l)} \equiv \partial L / \partial z_f^{(l)}$$

5.1 Step 1 — Output layer error, $\delta^{(3)}$

Chain rule through the output neuron:

$$\begin{aligned}
 \delta_1^{(3)} &= (\partial L / \partial y) \cdot (\partial y / \partial z_1^{(3)}) \\
 \delta_1^{(3)} &= [(y - t) / (y(1-y))] \cdot [y(1-y)]
 \end{aligned}$$

The $y(1-y)$ terms cancel — the classic sigmoid + binary-cross-entropy simplification:

$$\rightarrow \delta_1^{(3)} = y - t$$

5.2 Layer-3 weight gradients

$$\partial L / \partial w_{1,1}^{(3)} = \delta_1^{(3)} \cdot y_1^{(2)}$$

$$\partial L / \partial w_{2,1}^{(3)} = \delta_1^{(3)} \cdot y_2^{(2)}$$

$$\partial L / \partial w_{0,1}^{(3)} = \delta_1^{(3)}$$

5.3 Step 2 — Propagate error into layer 2, $\delta^{(2)}$

General rule: sum the incoming δ 's, weighted by the connecting weights, then multiply by the local sigmoid derivative:

$$\delta_j^{(2)} = (\sum_k \delta_k^{(3)} w_{j,k}^{(3)}) \cdot y_j^{(2)} (1 - y_j^{(2)})$$

With a single output neuron ($k = 1$), this gives:

$$\delta_1^{(2)} = \delta_1^{(3)} w_{1,1}^{(3)} \cdot y_1^{(2)} (1 - y_1^{(2)})$$

$$\delta_2^{(2)} = \delta_1^{(3)} w_{2,1}^{(3)} \cdot y_2^{(2)} (1 - y_2^{(2)})$$

5.4 Layer-2 weight gradients

$$\partial L / \partial w_{1,1}^{(2)} = \delta_1^{(2)} \cdot y_1^{(1)} \quad \partial L / \partial w_{2,1}^{(2)} = \delta_1^{(2)} \cdot y_2^{(1)} \quad \partial L / \partial w_{0,1}^{(2)} = \delta_1^{(2)}$$

$$\partial L / \partial w_{1,2}^{(2)} = \delta_2^{(2)} \cdot y_1^{(1)} \quad \partial L / \partial w_{2,2}^{(2)} = \delta_2^{(2)} \cdot y_2^{(1)} \quad \partial L / \partial w_{0,2}^{(2)} = \delta_2^{(2)}$$

5.5 Step 3 — Propagate error into layer 1, $\delta^{(1)}$

Same pattern, now summing over both layer-2 neurons that each layer-1 neuron feeds into:

$$\delta_j^{(1)} = (\sum_k \delta_k^{(2)} w_{j,k}^{(2)}) \cdot y_j^{(1)} (1 - y_j^{(1)})$$

$$\delta_1^{(1)} = (\delta_1^{(2)} w_{1,1}^{(2)} + \delta_2^{(2)} w_{1,2}^{(2)}) \cdot y_1^{(1)} (1 - y_1^{(1)})$$

$$\delta_2^{(1)} = (\delta_1^{(2)} w_{2,1}^{(2)} + \delta_2^{(2)} w_{2,2}^{(2)}) \cdot y_2^{(1)} (1 - y_2^{(1)})$$

5.6 Layer-1 weight gradients

$$\partial L / \partial w_{1,1}^{(1)} = \delta_1^{(1)} \cdot x_1 \quad \partial L / \partial w_{2,1}^{(1)} = \delta_1^{(1)} \cdot x_2 \quad \partial L / \partial w_{0,1}^{(1)} = \delta_1^{(1)}$$

$$\partial L / \partial w_{1,2}^{(1)} = \delta_2^{(1)} \cdot x_1 \quad \partial L / \partial w_{2,2}^{(1)} = \delta_2^{(1)} \cdot x_2 \quad \partial L / \partial w_{0,2}^{(1)} = \delta_2^{(1)}$$

6. General Recursion (Summary)

Every layer follows the same two-line recipe, working from the output backward:

$$\delta_j^{(l)} = (\sum_k \delta_k^{(l+1)} w_{j,k}^{(l+1)}) \cdot y_j^{(l)} (1 - y_j^{(l)}) \quad \leftarrow \text{for hidden layers}$$

$$\partial L / \partial w_{i,j}^{(l)} = \delta_j^{(l)} \cdot y_i^{(l-1)}, \quad \partial L / \partial w_{0,j}^{(l)} = \delta_j^{(l)}$$

with the output layer's δ computed directly as $y - t$ (the sigmoid + BCE shortcut), and every earlier layer's δ built from the next layer's δ values — hence “back”-propagation.

7. Gradient Descent Update

Once every δ and every weight gradient above is computed, each weight is updated using learning rate η :

$$w_{i,j}^{(l)} \leftarrow w_{i,j}^{(l)} - \eta \cdot \partial L / \partial w_{i,j}^{(l)}$$